

GITHUB-EN FUNTZIONAMENDU BURUZKO DOKUMENTAZIOA

2026. TXOSTENA

AMHIren
Garapen
Inguru
Lantaldea

T4



LEADING THE FUTURE

AURKIBIDEA

1. SARRERA	3
2. PRESTAKUNTZA ETA INSTALAZIOA	3
3. GIT BASH-EN KOMANDOEN ERABILERA	5
3.1 Klonatu eta identifikatu	5
3.2 Adarren kudeaketa	6
3.3 Garapena eta aldaketak gordetzea	7
3.4 Hodeiarekin eta GitHubekin sinkronizatzea	8
4. AURKITUTAKO ZAILTASUNAK	9
5. BIBLIOGRAFIA	10

1. SARRERA

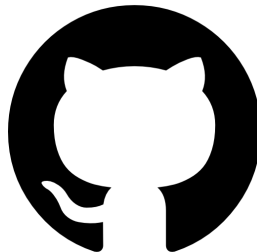
Dokumentazio horrek zehazten ditu gure taldeak jarraitu dituen **prozesu tekniko eta metodologikoak**, kodearen osotasuna ziurtatzeko, lankidetzaren denbora errealean errazteko eta gure aurrerapenaren historia gardena mantentzeko. Helburu horiek lortzeko, gure garapena funtsezko bi zutaberen gainean egituratu dugu: Git eta GitHub.

Zer da Git eta GitHub?

- **Git:** Gure **ordenagailuetan lokalean** erabiltzen dugun tresna da. Bere funtzio nagusia aldaketen historia osoa erregistratzea da, zerbaitek huts egiten badu "denboran zehar bidaiatzea" eta garapen-adar desberdinak kudeatzea ahalbidetuz.



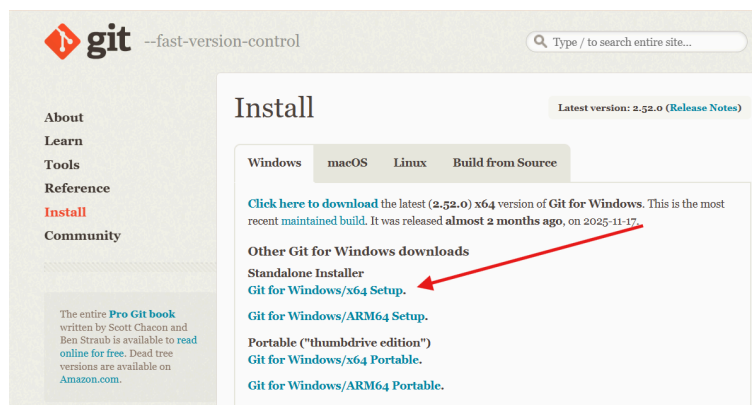
- **GitHub:** Proiektuaren **gordailua hartzen duen online** plataforma da. Talde osoaren lana zentralizatzeko, taldekide bakoitzaren aurrerapenak batzeko eta azken bertsioa irakasleari emateko balio digu.



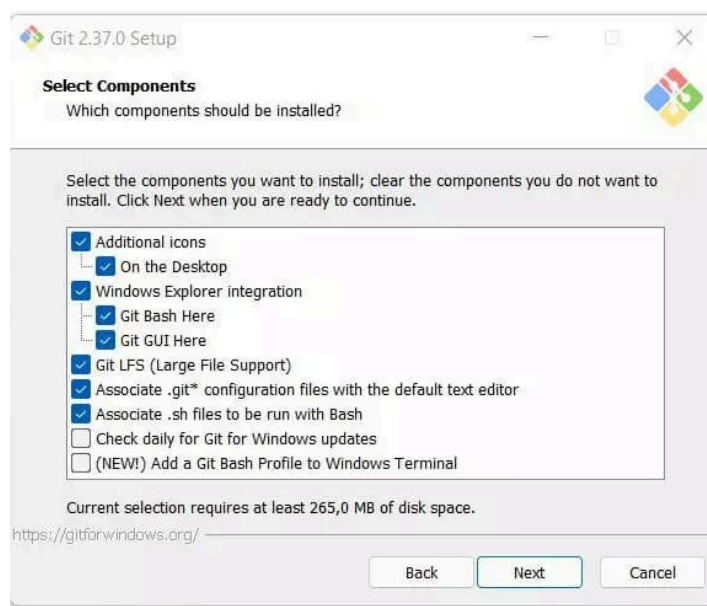
2. PRESTAKUNTZA ETA INSTALAZIOA

Lanean hasi baino lehen, **garapen-ingurunea konfiguratzeko** dugu urrats hauei jarraituz:

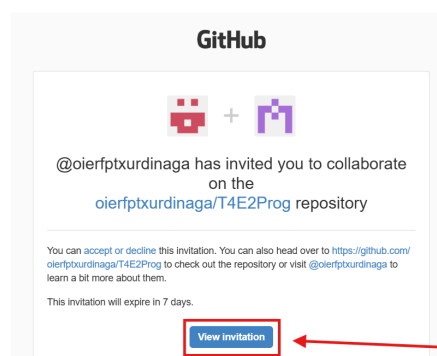
- **Git instalazioa:**
Softwarea deskargatu eta instalatuko dugu orrialde ofizialetik (<https://git-scm.com/install/windows>)



Gero instalazioan zehar **aurrez zehaztutako** aukerak utzita.



- **Proiektuarekin bat egin:**
Behin instalatuta, **GitHub-en** proiektura sartuko gara.

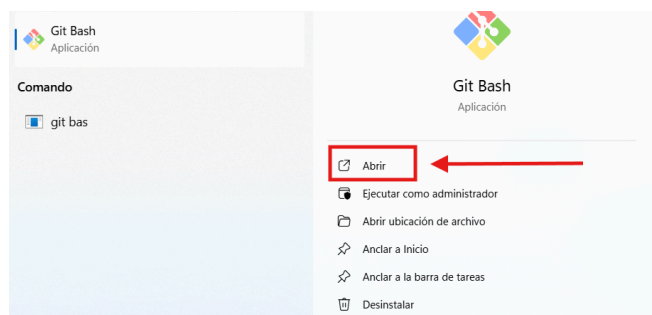


Hau amaiatuta, terminala (Git Bash edo CMD) erabiliko dugu proiektua hodeitik gure ordenagailu lokalera ekartzeko.

3. GIT BASH-EN KOMANDOEN ERABILERA

3.1 Klonatu eta identifikatu

Lehenik eta behin, **Git-en Bash** kotsola ireki behar genuen.



Behin irekita, **mahaigainera nabigatu behar** genuen, karpeta mahaigaineen eduki nahi dugu, **aldaketak egitea askoz errazagoa** izan dadin. Honetarako hau egin behar genuen:

```
364610F@HZ399418 MINGW64 ~  
$ cd Desktop/
```

- Taldearen **gordailua deskargatu** behar izan zen, eta gu identifikatzea, Gitek jakin dezan **nor ari den aldaketak** egiten. Honetarako komando hauek erlabili genituen:

- `$ git clone https://github.com/oierfptxurdinaga/T4E2MLDB`

```
364610F@HZ399418 MINGW64 ~/Desktop  
$ git clone https://github.com/oierfptxurdinaga/T4E2MLDB  
Cloning into 'T4E2MLDB'...  
remote: Enumerating objects: 3, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Receiving objects: 100% (3/3), done.
```

Git clone komandoak urruneko gordailu bat klonatzeko aukera ematen du ekipo lokalean, fitxategi guztiak eta proiektuaren historia deskargatuz, harekin lan egin ahal izateko.

- `ls`

```
364610F@HZ399418 MINGW64 ~/Desktop  
$ ls  
'CALENDARIO TODO EL CURSO.png' Martin_Hector/  
'Eclipse IDE for Java Developers - 2025-06.Ink' Martin_Hector.zip  
'Eclipse IDE for Java Developers - 2025-09.Ink' 'PLANIFIKAZIOA 1EB.png'  
'Erronka Version Final Hector' 'T4E2Prog/  
'FECHAS PRIMER CURSO.png' desktop.ini  
'HORARIO.png'
```

`Ls` komandoa erabili genuen egungo direktorioko fitxategiak eta karpetak zerrendatzeko eta proiektuaren karpeta behar bezala deskargatu zela egiaztatzeko.

Sortu berri den karpetan **sartuko gara (karpeta klonatua)**, proiektuari buruzko **Git aginduak** exekutatu ahal izateko.

```
364610F@HZ399418 MINGW64 ~/Desktop
$ cd T4E2MLDB/
```

- `$ git config --global user.name "Hector"`

```
364610F@HZ399418 MINGW64 ~/Desktop/T4E2MLDB (main)
$ git config --global user.name "Hector"
```

`Git config --global user.name` komandoa commiten egilearen izena definitzeko erabiltzen da, sistemako gordailu guztiei modu globalean aplikatuz.

- `$ git config --global user.mail "xxxxxxxxx"`

```
364610F@HZ399418 MINGW64 ~/Desktop/T4E2MLDB (main)
$ git config --global user.mail "h.martinsanchez@ikasle.eus"
```

`Git config --global user.email` komandoak commit-en egileari mezu elektroniko bat emateko balio du, eta modu globalean aplikatzen da sistemaren gordailu guztietan. Horrek ziurtatzen du aldaketa bakoitza behar bezala identifikatuta geratzen dela

3.2 Adarren kudeaketa

Bertsio nagusian zuzenean ez lan egiteko eta akatsak saihesteko, gure proiekturako lan-adar espezifiko bat sortu genuen.

- `$ git branch T4SPRINT1`

```
364610F@HZ399418 MINGW64 ~/Desktop/T4E2MLDB (main)
$ git branch T4SPRINT1
```

`Git branch 'xxxxxx'` komandoak adar berri bat sortzen du biltegian, garapen-lerro bereizi batean lan egiteko aukera emanez. Kasu

honetan, T4SPRINT1 funtzionalitate espezifikoak garatzeko sortu zen, adar nagusiari eragin gabe.

- \$ git branch

```
364610F@HZ399418 MINGW64 ~/Desktop/T4E2MLDB (main)
$ git branch
  T4SPRINT1
* main
```

Git branch tokiko adar guztien zerrenda erakusten digu, eta izartxo batekin (*) markatzen du non gauden kokatuta.

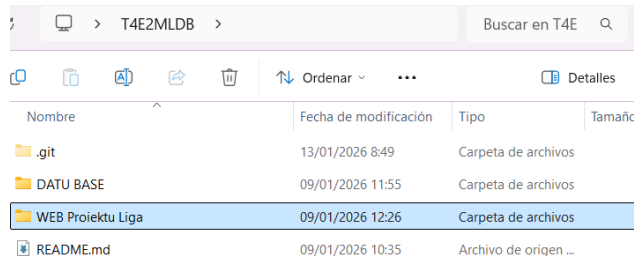
- \$ git checkout T4SPRINT1

```
364610F@HZ399418 MINGW64 ~/Desktop/T4E2MLDB (main)
$ git checkout T4SPRINT1
Switched to branch 'T4SPRINT1'
```

Git checkout adar nagusitik T4SPRINT1 adarrera mugitzen gaitu bertan lanean hasteko.

3.3 Garapena eta aldaketak gordetzea

Proiektuan fitxategiak aldatu eta sortu ondoren, gorde egin genituen.



Nombre	Fecha de modificación	Tipo	Tamaño
.git	13/01/2026 8:49	Carpeta de archivos	
DATU BASE	09/01/2026 11:55	Carpeta de archivos	
WEB Proiektu Liga	09/01/2026 12:26	Carpeta de archivos	
README.md	09/01/2026 10:35	Archivo de origen ...	

- \$ git status

```
364610F@HZ399418 MINGW64 ~/Desktop/T4E2MLDB (T4SPRINT1)
$ git status
On branch T4SPRINT1
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   DATU BASE/Datu Base Erronka 2 Dokumentazioa T4_.docx
    new file:   DATU BASE/Entitate erlazionala.drawio
    new file:   DATU BASE/Erronka 2 E-R.drawio
    new file:   DATU BASE/temporada_futbol.sql
```

Git status egiaztatzen du fitxategien egoera. Gorriz erakusten digu zein fitxategi aldatu diren, baina oraindik ez daude gordetzeko prestatuta.

- \$ git add WEB Proiektu Liga/

```
364610F@HZ399418 MINGW64 ~/Desktop/T4E2MLDB (T4SPRINT1)
$ git add WEB\ Proiektu\ Liga/
```

`Git add WEB Proiektu Liga/` prestatzen du (stage) zehazki web-proiektuaren karpeta hurrengo gordetzerako.

- `$ git add *`

```
364610F@HZ399418 MINGW64 ~/Desktop/T4E2MLDB (T4SPRINT1)
$ git add *
warning: in the working copy of 'DATU BASE/Entitate erlazionala.drawio', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'DATU BASE/Erronka 2 E-R.drawio', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'DATU BASE/temporada_futbol.sql', LF will be replaced by CRLF the next
time Git touches it
```

`Git add *` prestatzen du aldatutako fitxategi guztiak edo berriak egungo direktorioan (dena bat-batean gehitzeko erabilgarria).

- `$ git commit -m "xxxxx"`

```
364610F@HZ399418 MINGW64 ~/Desktop/T4E2MLDB (T4SPRINT1)
$ git commit -m "Datu basearen lehenengo bertsioak"
[T4SPRINT1 84f7e0b] Datu basearen lehenengo bertsioak
Committer: Hector <364610F@H015110.NET>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly:

    git config --global user.name "Your Name"
    git config --global user.email you@example.com

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

4 files changed, 1337 insertions(+)
create mode 100644 DATU BASE/Datu Base Erronka 2 Dokumentazioa T4_.docx
create mode 100644 DATU BASE/Entitate erlazionala.drawio
create mode 100644 DATU BASE/Erronka 2 E-R.drawio
create mode 100644 DATU BASE/temporada_futbol.sql
```

`Git commit -m "xxxxx"` prestatutako aldaketak berristen ditu eta "argazki" bat (bertsioa) sortzen du tokiko historialean, mezu deskribatzaile batekin.

3.4 Hodeiarekin eta GitHubekin sinkronizatzea

Lana ordenagailuan gorde ondoren, zerbitzariari bidali eta bateratu egin genuen. Honetarako bi komandu erabili genuen:

- `$ git push-u origin T4SPRINT1`

```
364610F@HZ399418 MINGW64 ~/Desktop/T4E2MLDB (T4SPRINT1)
$ git push -u origin T4SPRINT1
Enumerating objects: 55, done.
Counting objects: 100% (55/55), done.
Delta compression using up to 4 threads
Compressing objects: 100% (51/51), done.
Writing objects: 100% (54/54), 3.45 MiB | 2.35 MiB/s, done.
Total 54 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), done.
remote:
remote: Create a pull request for 'T4SPRINT1' on GitHub by visiting:
remote:   https://github.com/oierfptxurdinaga/T4E2MLDB/pull/new/T4SPRINT1
remote:
To https://github.com/oierfptxurdinaga/T4E2MLDB
 * [new branch]      T4SPRINT1 -> T4SPRINT1
branch 'T4SPRINT1' set up to track 'origin/T4SPRINT1'.
```

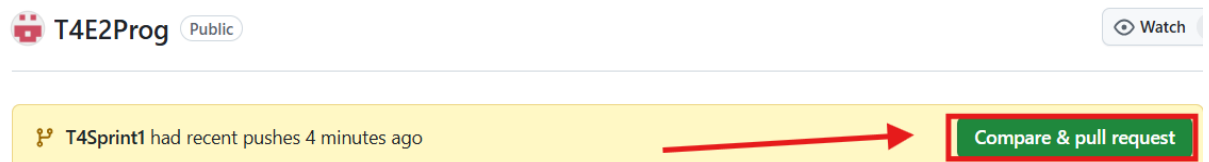

`Git push-u origin T4SPRINT1` komandoa gure T4SPRINT1 tokiko adarreko commit-ak GitHub-eko biltegira (jatorria) igotzen du. - u aukerak etorkizuneko bidalketetarako esteka ezartzen du.

- `$ git pull`

```
364610F0HZ399418 MINGW64 ~/Desktop/T4E2MLDB (T4SPRINT1)
$ git pull
Already up to date.
```

`Git pull` komandoa deskargatzen du eta fusionatzen du beste lankide batzuek GitHub-era igo dituzten aldaketa berrienak, gure biltegi lokala eguneratuz.

Pull Request (GitHub-en webgunean): Push-a egin ondoren, GitHub-en orrira joan ginen. Han ohar bat agertu zen "Konparatu eta Pull Request sortzeko".



Ondoren, **egindako aldaketak** deskribatuko genituen eta **"Create Pull Request"** sakatu genuen.

Amaitzeko, kodea berrikusi eta Merge onartu zen, gure T4SPRINT1 adarra proiektuaren **adar nagusiarekin (main)** bat eginez.

4. AURKITUTAKO ZAILTASUNAK

Garapenean zehar, **gatazka komun** bat aurkitzen dugu `git push` bat egiten saiatzean.

- **Arazoa:** kontsolak uko egin zion komandoari, urrutiko biltegiak lokalean ez genuen lana zuela adieraziz (**rejected - non-fast-forward**). Hori gertatu zen **lankide batek aldaketak** egin zituelako gu lanean ari ginen bitartean.
- **Irtenbidea:** Lehendabizi **git pull** bat exekutatu behar izan genuen **azken bertsio horiek deskargatzeko** eta gure lanarekin fusionatzeko. Behin gure tokiko proiektua eguneratuta, arazorik gabe egin ahal izan genuen git push delakoa.

5. BIBLIOGRAFIA

Prozesu hau egiteko eta komandoak ikasteko, honako baliabide hau erabili dugu erreferentzia gisa:

Git (tutoriala): https://www.youtube.com/watch?v=ky77PUkO_jQ